

Laboratorio #5

Navegadores: Rendimiento

DevTools

Las **herramientas para desarrolladores** permite depurar, debuggear, ver más detalles del desarrollo, atributos de css, html, JavaScript e incluso evaluar la velocidad al evaluar llamadas y tiempos de respuestas por request, entre otras utilidades. Estas pueden tener diversos modos de acceso dependiendo del navegador.

Acción	Utilidad	Chrome	Firefox	Opera
Inspeccionar HTML	Permite visualizar y modificar el código fuente de una página en tiempo real, útil para desarrollo y depuración.	Ctrl + Shift + I → pestaña 'Elements'	Ctrl + Shift + I → pestaña 'Inspector'	Ctrl + Shift + I → pestaña 'Elements'
Inspeccionar DOM	Muestra la estructura del DOM, permitiendo modificar elementos, atributos y eventos.	Herramientas de desarrollador → 'Elements'	Herramientas de desarrollador → 'Inspector'	Herramientas de desarrollador → 'Elements'
Inspeccionar CSS	Permite visualizar y modificar propiedades CSS en tiempo real, ayudando en el diseño y ajustes de estilos.	Herramientas de desarrollador → 'Styles'	Herramientas de desarrollador → 'Rules'	Herramientas de desarrollador → 'Styles'
Inspeccionar CSSOM	Muestra el Modelo de Objetos CSS, útil para inspeccionar la aplicación dinámica de estilos.	Herramientas de desarrollador → Consola	Herramientas de desarrollador → Consola	Herramientas de desarrollador → Consola
Inspeccionar Layout	Permite ver la distribución de los elementos en la página, útil para ajustes de diseño responsivo.	Herramientas de desarrollador → 'Layout'	Herramientas de desarrollador → 'Box Model'	Herramientas de desarrollador → 'Layout'
Inspeccionar Árbol de renderizado	Muestra cómo el navegador genera la representación visual de la página, útil para optimización de rendimiento.	Herramientas de desarrollador → 'Performance'	Herramientas de desarrollador → árbol Dom y estilos aplicados	Herramientas de desarrollador → 'Performance'
Debugger de JavaScript	Permite analizar y corregir errores en el código JavaScript mediante puntos de interrupción y seguimiento de ejecución.	Herramientas de desarrollador → 'Sources' → 'Debugger'	Herramientas de desarrollador → 'Debugger'	Herramientas de desarrollador → 'Sources'

Los **puntos de interrupción** (breakpoints) que se pueden utilizar en herramientas de depuración de navegadores como Chrome, Firefox y Opera; se usan para analizar la ejecución del código JavaScript y otros aspectos de la web, pausando el programa en momentos específicos para inspeccionar su comportamiento

Tipo de punto de interrupción	Ejemplo en JavaScript	Explicación
Línea de código	<code>debugger; console.log('Pausa aquí');</code>	Pausa la ejecución del código en la línea exacta donde se encuentra 'debugger'.
Línea de código condicional	<code>if (x > 10) { debugger; }</code>	Detiene la ejecución solo si 'x' es mayor que 10.
Punto de registro	<code>console.log('Valor de x:', x);</code>	Muestra un mensaje en la consola sin detener el código.
DOM	<code>document.getElementById('boton').remove();</code>	Pausa la ejecución cuando se elimina el nodo con id='boton'.
XHR	<code>fetch('https://api.example.com/data').then(response => console.log(response));</code>	Pausa cuando una solicitud coincide con un patrón de URL específico.
Objeto de escucha de eventos	<code>document.addEventListener('click', () => console.log('Clic detectado'));</code>	Pausa cuando se activa el evento 'click' en el documento.
Excepción	<code>try { throw new Error('Algo salió mal'); } catch (e) { console.error(e.message); }</code>	Pausa cuando ocurre una excepción, ya sea detectada o no detectada.
Función	<code>function saludar() { console.log('Hola!'); } saludar();</code>	Pausa cada vez que se ejecuta 'saludar()'.
Tipo de confianza	Dependiente del navegador	Relacionado con optimización y seguridad en navegadores modernos.

- **Evaluación del Proceso de Diseño con DevTools:**

- **Composición:** Inspecciona el diseño con la herramienta de elementos (Elements) en DevTools. Observa la estructura de los contenedores (divs), alineación y distribución de los elementos.
- **Jerarquía visual:** Usa la pestaña de Styles para analizar el tamaño, contraste y ubicación de los elementos. Identifica cuáles tienen mayor peso visual mediante font-size, color, z-index y otros atributos.
- **Tipografía:** Examina las fuentes aplicadas con el inspector de CSS en DevTools. Evalúa si la tipografía es legible y armoniza con el resto del diseño.

- **Análisis del uso del color:**

- **Paleta cromática:** Con DevTools, revisa los estilos de los elementos (background-color, color) y utiliza la herramienta de Color Picker para inspeccionar los valores hexadecimales.
- **Psicología del color:** Identifica la intención del diseñador observando cómo se aplican los colores en secciones específicas y su efecto en la experiencia del usuario.
- **Contrastes:** Usa la herramienta de accesibilidad en DevTools (Lighthouse) para verificar si el contraste entre colores cumple con las normativas WCAG.

- **Técnica y ejecución pictórica:**

- **Texturas y materiales:** Si hay imágenes o fondos con efectos visuales, inspeccionarlos con la herramienta Network para ver su formato y calidad.
- **Uso de la luz:** Analiza sombras y reflejos con la inspección de CSS (box-shadow, filter: drop-shadow).
- **Perspectiva y proporción:** Revisa los atributos de transformación (transform: scale, perspective) para entender la profundidad y tridimensionalidad.

Usar DevTools te permitirá obtener datos técnicos y objetivos sobre el diseño, asegurando que los principios visuales y de accesibilidad sean correctamente aplicados.

Bonus

¿Cuál fue la experiencia con el reto 4? ¿Algún cambio detectado que generó cambios en el proyecto? Consigue un bono en tu calificación del reto 4 presentando en par de minutos tu experiencia a tus compañeros.

Ejercicios

- a) Uso de this en JavaScript

Comprender cómo cambia el comportamiento de this en diferentes contextos de JavaScript, recordemos de la teoría:

Contexto	Explicación
Ámbito global	this apunta al objeto window en navegadores.
Función normal	this depende de cómo se llame; en modo estricto es undefined, en modo normal es window.
Método de objeto	this apunta al propio objeto que ejecuta el método.
Función flecha	No tiene su propio this, sino que hereda el del contexto donde fue definida.

1. Crea un archivo *this_practica.html*. Coloca este que código incluye ejemplos de **this** en diferentes escenarios.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Ejemplo de "this" en JavaScript</title>
</head>
<body>
  <h1>Práctica de "this" en JavaScript</h1>
  <button onclick="mostrarThis()">Ejecutar Función Normal</button>
  <button onclick="objeto.mostrarNombre()">Ejecutar Método de
Objeto</button>
  <button onclick="objetoFlecha.mostrarNombre()">Ejecutar Arrow
Function</button>

  <script>
    // ① ÁMBITO GLOBAL
    console.log("Ámbito global:", this); // En un navegador, `this`
es `window`

    // ② FUNCIÓN NORMAL
    function mostrarThis() {
      "use strict"; // Activa modo estricto
      console.log("Función normal:", this); // En modo estricto,
`this` es `undefined`, en no estricto es `window`
    }

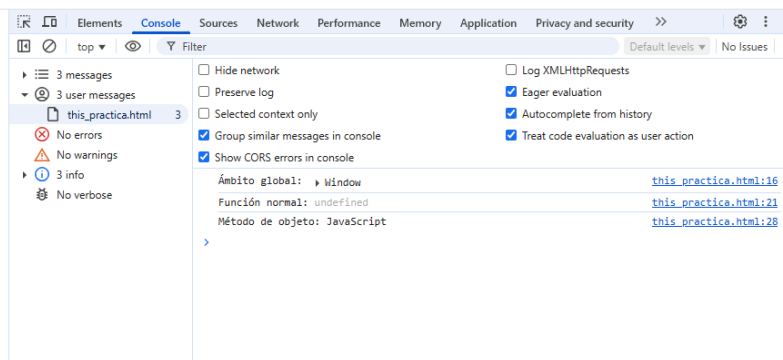
    // ③ MÉTODO EN UN OBJETO
    const objeto = {
      nombre: "JavaScript",
      mostrarNombre: function() {
        console.log("Método de objeto:", this.nombre); // Apunta
al objeto
      }
    };
  </script>
</body>
</html>
```

```
// ④ FUNCIÓN FLECHA
const objetoFlecha = {
  nombre: "JavaScript",
  mostrarNombre: () => {
    console.log("Arrow function:", this.nombre); // Hereda
    `this` del contexto global
  }
};
</script>
</body>
</html>
```

2. Ejecución del archivo

- Guarda el archivo como *this_practica.html*.
- Ábrelo en un navegador con *live server*.
- Abre la consola del navegador: En Chrome/Firefox presiona F12 y ve a la pestaña Console.

Práctica de "this" en JavaScript



3. Interacción y análisis

- Observa el valor de *this* en la consola antes de hacer clic en los botones.
- Haz clic en los botones de la página:
 - Ejecutar Función Normal:** Evalúa qué sucede con *this* dentro de una función regular.
 - Ejecutar Método de Objeto:** Observa cómo *this* apunta al objeto que contiene la función.
 - Ejecutar Arrow Function:** Nota que *this* en una función flecha no cambia y mantiene el valor del contexto exterior.
- Depuración: Pausar la ejecución para observar *this* en cada caso:
 - Usa debugger; dentro de las funciones:

```
function mostrarThis() {
  debugger;
  console.log(this);
}
```

Al ejecutar la función, la ejecución se detendrá en esa línea y podrás inspeccionar *this*.

4. Interacción y análisis. Comparte con tus compañeros o preparador tus impresiones sobre las siguientes preguntas:
 - i. ¿Se logró identificar qué contexto afecta `this`?
 - ii. ¿Se diferenció el comportamiento en función normal, método de objeto y función flecha?
 - iii. ¿Cuál es la utilidad del `console.log()`?
 - iv. ¿Cuál consideras es utilidad de esta herramienta particular para inspeccionar en el código funciones y variables según el contexto?

5. Explicación de esta práctica

```
Ámbito global: this_practica.html:16
  ▶ Window {window: Window, self: Window, document: document, name: '', Location: Location, ...}
> console.log("Ámbito global:", this);
Ámbito global: VM192:1
  ▶ Window {window: Window, self: Window, document: document, name: '', Location: Location, ...}
< undefined
5 Función normal: undefined this_practica.html:21
5 Método de objeto: JavaScript this_practica.html:28
5 Arrow function: undefined this_practica.html:36
>
```

Aquí tienes la explicación de los cuatro casos sobre el comportamiento de `this` en JavaScript:

- **Ámbito global**
 - En un navegador, `this` en el ámbito global hace referencia al objeto `window`.
 - Puedes verlo simplemente escribiendo `this` en la consola sin ejecutar código adicional.
- **Función normal**
 - Si una función normal se ejecuta sin estar dentro de un objeto, `this` dependerá de si se usa `"use strict"` o no.
 - Sin modo estricto, `this` apunta a `window`.
 - Con modo estricto, `this` será `undefined`, ya que el modo estricto evita que `this` tome el valor global por defecto.
- **Método dentro de un objeto**
 - Cuando `this` se usa dentro de una función definida dentro de un objeto (método), `this` apunta al propio objeto.
 - Esto permite acceder a las propiedades del objeto dentro del método.
- **Función flecha**
 - Las funciones flecha no tienen su propio `this`, sino que heredan el valor del contexto en el que fueron definidas.
 - Si la función flecha está dentro de un objeto, `this` no apunta al objeto, sino al contexto global (`window` en navegadores).
 - Esto puede causar que el acceso a propiedades del objeto falle, mostrando `undefined`.

En este código se activó el modo estricto, pero solo dentro de la función `mostrarThis()`. El modo estricto en JavaScript es una característica que se activa con `"use strict"`; y ayuda a escribir código más seguro y depurado. Cuando está activado, impone reglas más estrictas sobre el uso de variables y funciones, evitando errores comunes o prácticas problemáticas.

- ¿Qué cambia con el modo estricto?

- **Evita el uso de this no definido** → En funciones normales, **this** será **undefined** en lugar de **window**.
- **Bloquea variables sin declarar** → No puedes asignar valores a variables que no han sido declaradas con **let**, **const** o **var**.
- **Previene la eliminación de variables y funciones** → No puedes hacer **delete variable**; o **delete function()**;
- **Evita nombres duplicados en parámetros** → No puedes repetir nombres de parámetros en una función.
- **Deshabilita el uso de palabras reservadas** → Algunas palabras como **eval** y **arguments** tienen restricciones adicionales.

b) Optimizar CSS

Exploramos cómo los selectores CSS pueden impactar el rendimiento de una página web, específicamente durante los eventos **Recalculate Style**. A lo largo de esta práctica, identificamos los selectores que consumen más recursos y aprenderemos a optimizarlos.

1. Construcción de la Página Inicial. Para comenzar, copia y pega el siguiente código en un archivo **index.html** (version 1):

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Optimización de Selectores CSS</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <header>
    <h1>Optimización de Selectores CSS</h1>
  </header>
  <main>
    <section class="content">
      <div class="box">
        <p class="text">Este es un párrafo dentro de un div.</p>
      </div>
      <div class="box">
        <p class="text">Otro párrafo dentro de otro div.</p>
      </div>
    </section>
  </main>
</body>
</html>
```

Ahora, copia y pega el siguiente código en un archivo **styles.css** (Versión 1):

```
/* Estilos iniciales */  
body {  
    font-family: Arial, sans-serif;  
}  
.content div p {  
    color: blue;  
    font-size: 18px;  
}
```

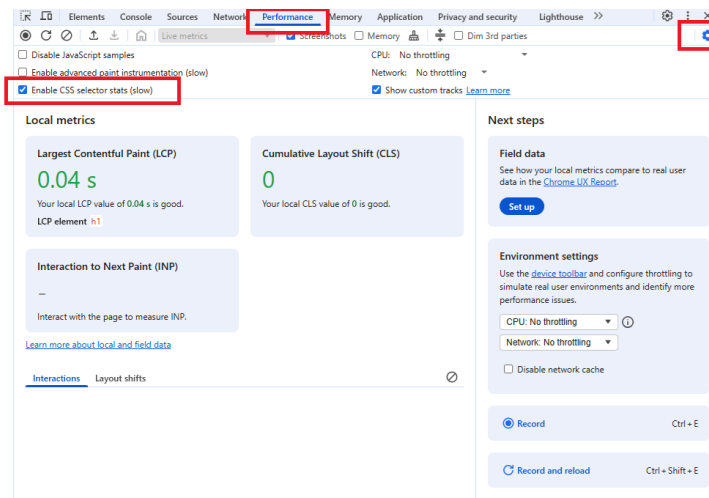
2. Grabación del Rendimiento en DevTools

- Accede a las Herramientas para Desarrolladores:
 - i. Abre tu página en Google Chrome.
 - ii. Presiona F12 o Ctrl + Shift + I en Windows (Cmd + Option + I en Mac).
 - iii. Dirígete a la pestaña **Performance**.

Optimización de Selectores CSS

Este es un párrafo dentro de un div.

Otro párrafo dentro de otro div.

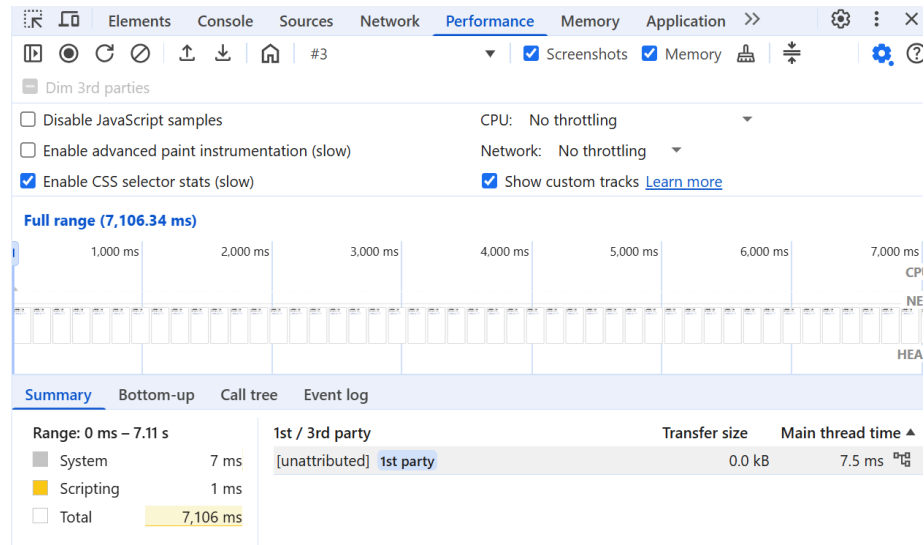


- Activa las estadísticas de selectores:
 - i. Haz clic en el ícono de configuración ⚙️.
 - ii. Marca la opción Enable CSS selector stats.
- Graba el rendimiento:
 - i. Haz clic en Record.
 - ii. Interactúa con la página (agrega/quita elementos, modifica clases).
 - iii. Detén la grabación con Stop.
- 3. Observación del primer escenario
 - i. Busca los eventos Recalculate Style.
 - ii. Abre la pestaña Selector Stats y observa qué selectores consumen más tiempo.
 - iii. Toma una captura del escenario para tener el historial del comportamiento.

Optimización de Selectores CSS

Este es un párrafo dentro de un div.

Otro párrafo dentro de otro div.



4. Optimización de Selectores CSS

Para mejorar el rendimiento, ajusta el código en `styles.css` (versión 2) de la siguiente manera:

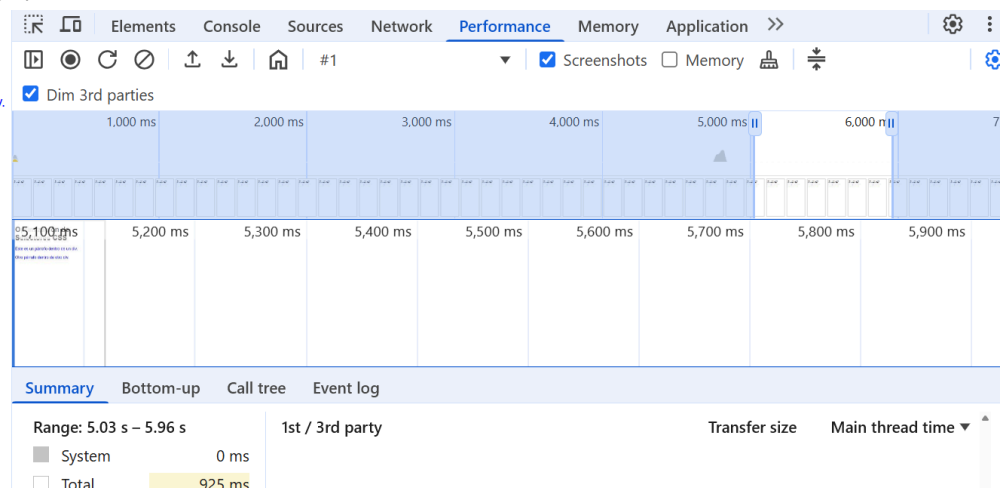
```
/* Evitar selectores descendientes costosos */  
.box p {  
  color: blue;  
  font-size: 18px;  
}
```

Ahora, repite la grabación del rendimiento y compara los tiempos de cálculo antes y después de la optimización.

Optimización de Selectores CSS

Este es un párrafo dentro de un div.

Otro párrafo dentro de otro div.



5. Analizar y concluir. Toma una captura de pantalla para tus registros y comparte con tus compañeros o preparador:

- ¿Lograste identificar los selectores que tuvieron un impacto significativo en el rendimiento?
- ¿Cómo los cambios afectaron el desempeño de la página?
- ¿Cómo consideras que fue mejorando el código CSS?

6. Explicación de Optimización aplicada:

Esta optimización del CSS se centró en mejorar la eficiencia de los selectores CSS para reducir el impacto en el rendimiento del navegador durante los eventos Recalculate Style.

- **Evitar selectores CSS demasiado específicos**

Antes:

```
.content div p {  
  color: blue;  
  font-size: 18px;  
}
```

Este selector obliga al navegador a recorrer varios niveles del DOM (.content > div > p) para aplicar estilos, lo que aumenta el tiempo de cálculo.

Después:

```
.box p {  
  color: blue;  
  font-size: 18px;  
}
```

Se usa un selector más directo, evitando la necesidad de recorrer múltiples niveles, lo que mejora el tiempo de procesamiento de estilos.

- **Reducción del impacto en el proceso de renderizado**

Se eliminan reglas de estilo innecesarias que podrían generar cálculos adicionales.

- **Evitar la recalculación masiva de estilos**

Los selectores optimizados permiten que el navegador procese menos elementos en cada actualización del DOM, lo que reduce los eventos Recalculate Style.

- **Verificación de mejoras**

Después de la optimización, volver a grabar el rendimiento en DevTools y comparar los tiempos de cálculo antes y después de los cambios nos permite comparar los escenarios.

c) Optimizar uso de CPU

Vamos a modificar el archivo index.html (versión 2) para incluir animaciones y generar un entorno ideal para evaluar el rendimiento de la página.

1. Actualiza el archivo *index.html* y reemplázalo con el siguiente código:

```
<!DOCTYPE html>  
<html lang="es">  
<head>
```

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width,
initial-scale=1.0">
<title>Prueba de Rendimiento CSS y JS</title>
<link rel="stylesheet" href="styles.css">
</head>
<body>
  <header>
    <h1>Prueba de Rendimiento</h1>
    <button id="addElement">Agregar 10 elementos</button>
    <button id="removeElements">Quitar 10 elementos</button>
    <button id="optimize">Optimizar</button>
    <button id="deoptimize">Desactivar Optimización</button>
  </header>
  <main id="container">
    <!-- Aquí se generarán los cuadrados azules dinámicamente -->
  </main>
  <script src="script.js"></script>
</body>
</html>
```

2. Modificar el archivo styles.css (versión 3). Ahora, abre el archivo styles.css y reemplázalo con el siguiente código:

```
body {
  font-family: Arial, sans-serif;
  text-align: center;
}

button {
  margin: 10px;
  padding: 10px;
  font-size: 16px;
}

#container {
  display: flex;
  flex-wrap: wrap;
  justify-content: center;
  margin-top: 20px;
}

.box {
  width: 50px;
```

```
height: 50px;
background-color: blue;
margin: 5px;
transition: transform 0.2s;
}

/* Versión optimizada */
.optimized .box {
  will-change: transform;
}
```

3. Agregar un archivo JavaScript. Crea un nuevo archivo llamado *script.js* en la misma carpeta y pega el siguiente código:

```
const container = document.getElementById("container");
const addBtn = document.getElementById("addElements");
const removeBtn = document.getElementById("removeElements");
const optimizeBtn = document.getElementById("optimize");
const deoptimizeBtn = document.getElementById("deoptimize");

function addElements() {
  for (let i = 0; i < 10; i++) {
    const box = document.createElement("div");
    box.classList.add("box");
    container.appendChild(box);
  }
}

function removeElements() {
  for (let i = 0; i < 10; i++) {
    if (container.lastChild) {
      container.removeChild(container.lastChild);
    }
  }
}

function optimize() {
  container.classList.add("optimized");
}

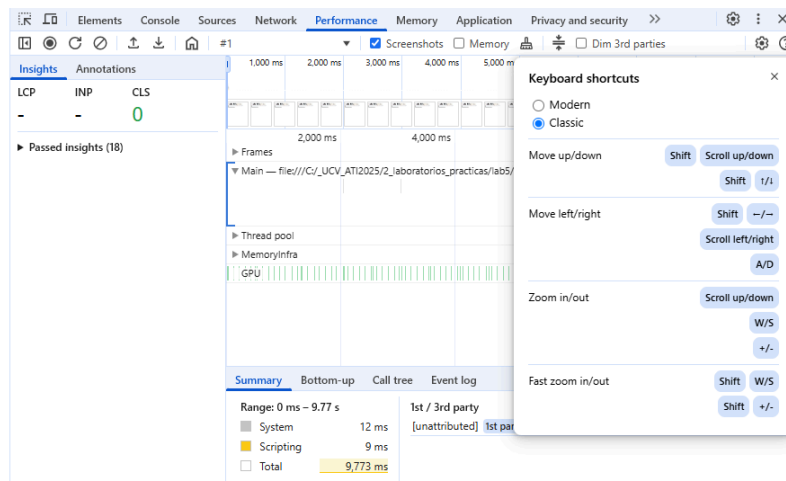
function deoptimize() {
  container.classList.remove("optimized");
}
```

```
addBtn.addEventListener("click", addElements);  
removeBtn.addEventListener("click", removeElements);  
optimizeBtn.addEventListener("click", optimize);  
deoptimizeBtn.addEventListener("click", deoptimize);
```

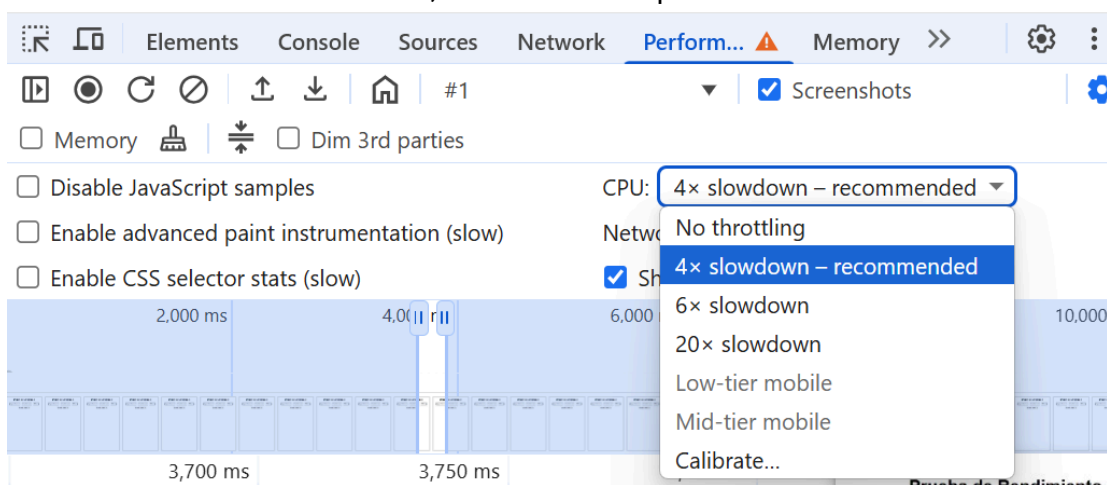
4. Analizar el rendimiento en DevTools. Sigue estos pasos para evaluar cómo los elementos dinámicos afectan el rendimiento:
 - Acceder a DevTools:
 - i. Abre la página en Google Chrome.
 - ii. Presiona F12 o Ctrl + Shift + I en Windows (Cmd + Option + I en Mac).
 - iii. Ve a la pestaña "Performance".
 - iv. Activar la captura de pantalla: Marca la casilla Capturas de pantalla en DevTools.

Prueba de Rendimiento

Agregar 10 elementos Quitar 10 elementos Optimizar
Desactivar Optimización

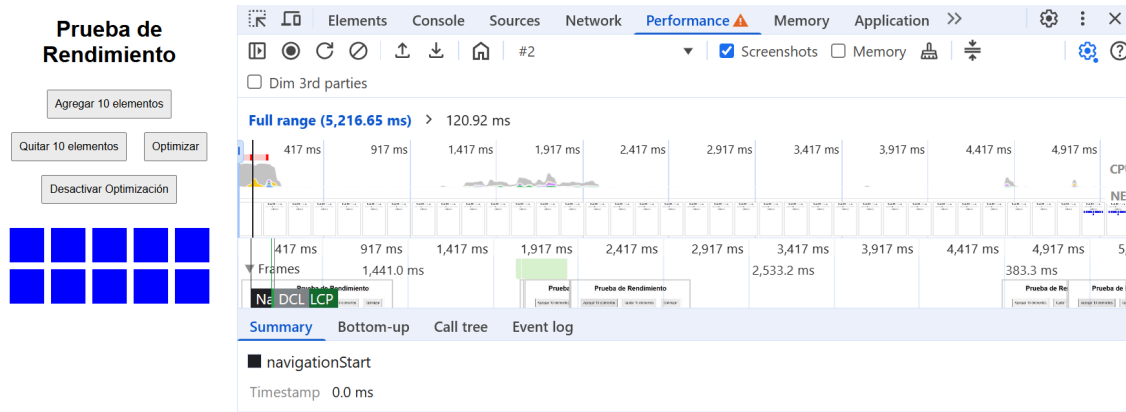


- Simular una CPU móvil:
 - i. Haz clic en Configuración de captura
 - ii. En la sección CPU, selecciona Multiplicador $\times 4$.



- Grabar el rendimiento:

- i. Haz clic en "Record" y presione los botones de la página (Agregar 10, Optimizar, Desactivar Optimización).
 - ii. Detén la grabación con "Stop".
- Analizar los resultados:
 - i. Revisa el gráfico de FPS y observa si aparecen barras rojas.
 - ii. Mueve el cursor sobre el gráfico de CPU para ver si la CPU está alcanzando su límite.
 - iii. Compara la diferencia entre la versión con CPU del PC y la simulación móvil.



5. Analizar y concluir. Comparte con tus compañeros o preparador:
 - i. ¿En qué momentos en los que la CPU alcanzó su límite?
 - ii. ¿Cómo los cambios afectaron la fluidez de los elementos animados?
 - iii. Propón mejoras en la optimización que se te ocurran.

6. Analizar y concluir. Comparte con tus compañeros o preparador:

La optimización en styles.css en este ejercicio consiste en mejorar el rendimiento de los estilos CSS para reducir el impacto en el tiempo de ejecución. En este laboratorio, aplicamos los siguientes principios de optimización:

- **Reducir selectores costosos:**

Eliminamos selectores descendientes excesivos (`.content div p`) y usamos selectores más eficientes:

```
.box p {  
  color: blue;  
  font-size: 18px;  
}
```

Esto evita que el navegador tenga que recorrer múltiples niveles del DOM para aplicar el estilo.

- **Utilizar will-change en animaciones**

Añadimos `will-change: transform;` a los elementos animados, lo que ayuda al navegador a preprocesar la animación y mejorar su fluidez:

```
.optimized .box {  
  will-change: transform;  
}
```

Esto reduce el tiempo de cálculo y mejora la velocidad de las animaciones.

- **Evitar repintados innecesarios**

En lugar de cambiar propiedades que afectan el tamaño y ubicación del elemento (como width o top), usamos transformaciones (transform: translateX();) para animaciones más rápidas.

- **Minimizar la cantidad de elementos afectados**

Los estilos optimizados reducen la cantidad de elementos que requieren cambios en el renderizado, lo que disminuye la carga en la CPU.

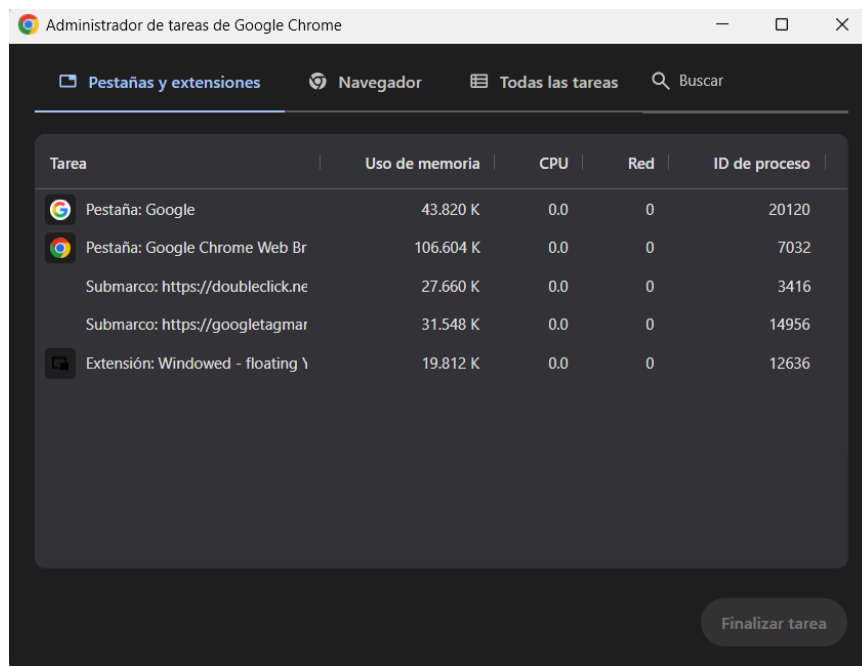
La combinación de estas mejoras permite que la página funcione de manera más eficiente y fluida, especialmente en dispositivos con menos potencia.

d) Extra: Acceder al administrador de tareas del navegador.

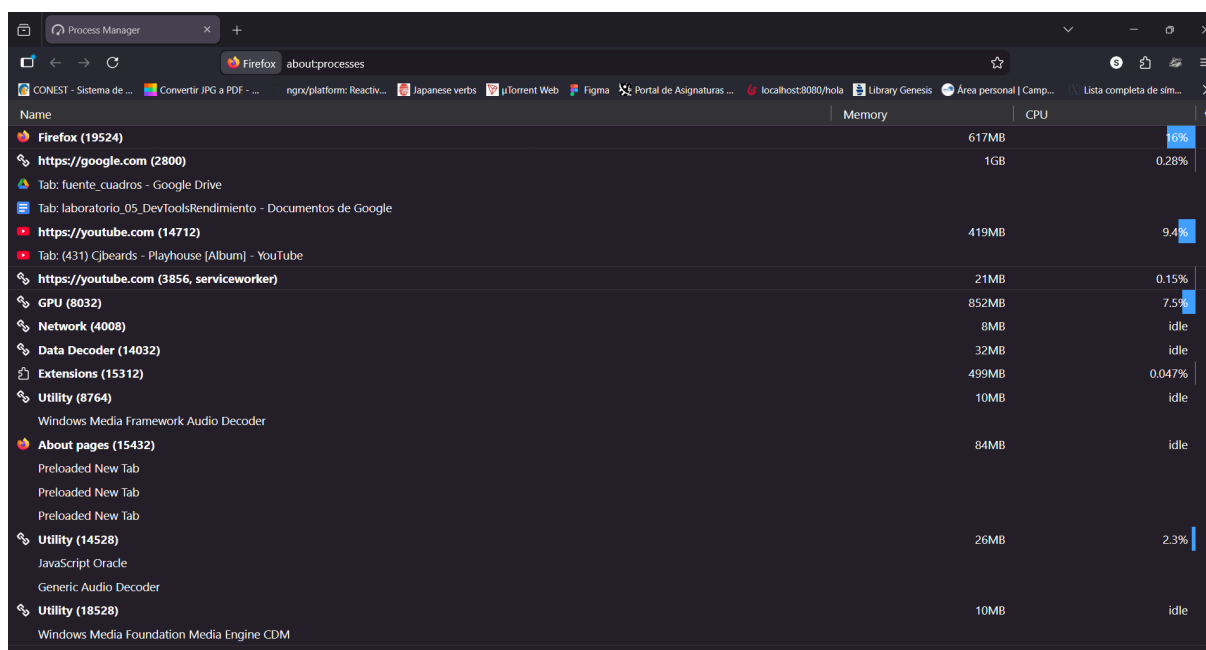
Otra herramienta a nuestra disposición para analizar el rendimiento de una aplicación web es consultar el **Administrador de Tareas** de nuestro navegador. Al igual que el que podemos encontrar en nuestro sistema operativo, este nos permite ver los procesos activos en el navegador, los recursos que emplean en tiempo real y los hilos que levantan.

Acceder al administrador de tareas dependerá del navegador que empleemos, sin embargo el proceso es parecido tanto para Firefox y Chrome:

1. Hacer clic en el menú en la esquina superior izquierda de la ventana del navegador.
2. Seleccionar la opción "More Tools" o "Más Herramientas".
3. Escoger "Task Manager" o "Administrador de tareas"



Administrador de Tareas: Chrome



Administrador de Tareas: Firefox



Reto 5 (at home)

En el reto previo se analizó la página *index.html*, en esta oportunidad, usando los nuevos conocimientos adquiridos, analizaremos el comportamiento de la página *profile.html* de su proyecto:

1. Identifica en tu código el tipo de uso de `this`:

- Ubique en los JavaScript del proyecto 3 funciones o uso de `var` - `let` - `const` referenciadas con `this`.
- Realice el análisis con DevTools.
- Capture la imagen de la prueba (imagen 1.a x 3).
- Determine qué tipo es según su contexto, justifique.

Si su código no tiene suficientes usos de `this` originalmente, puede incluir usos con `console.log()`

2. Analice el rendimiento del selector de CSS:

- Identifica los selectores consumen más tiempo realizando el análisis con DevTools.
- Haz una captura de pantalla (imagen 2.a) de la prueba del estado actual.
- Optimización de ejecución: añada a tu proyecto una mejora de los selectores CSS si identificas algún ajuste en código según lo aprendido en el laboratorio.
- Si aplica: realiza un captura de pantalla (imagen 2.b) de la prueba del estado posterior a la actualización.
- Justifique en tres líneas los cambios realizados en el CSS.

3. Analiza la aplicación

- Realiza la simulación de CPU móvil: reduce la velocidad de procesamiento para evaluar cómo funciona la página en dispositivos de menor rendimiento.
- Registra el rendimiento: Usa la pestaña Performance en DevTools para capturar métricas de ejecución. Toma una impresión de pantalla del monitoreo (imagen 3.a).
- Análisis de FPS y uso de CPU: Identifica algún cuello de botella en la renderización.
- Optimización de ejecución: Si aplica, ajusta el código para mejorar la fluidez de la página y reducir el consumo de recursos, coloca tus cambios en la rama del reto.
- Si realiza algún cambio: capture la imagen de la prueba después del mismo (imagen 3.b).
- Justifique en tres líneas dicha actualización, cuál fue la mejora y por qué.



4. Coloque los insumos obtenidos en un pdf (justificaciones de ejercicio 1, 2 y 3, con respectivas imágenes), membretado con sus datos, con nombre:

reto05_<CI>_<NombreApellido> ; ejemplo: reto05_13852255_BettyTorres.pdf.

Añada este archivo al directorio "docs"

Al terminar, colocar en el reto del Aula Virtual el url del repositorio de GitHub que uso para hacer este ejercicio y una imagen del historial de Git Graph que muestra su Visual Code.